MLOPS EXPERIMENTAL LEARNING ASSIGNMENT

Heart Disease Risk Prediction End-to-End MLOps Pipeline

Model development, experiment tracking, CI/CD, containerization, Kubernetes deployment and monitoring

Dataset	UCI Heart Disease (Cleveland), 303 patients, 13 features
Final model	Logistic Regression (C = 0.1), CV ROC-AUC 0.911, test ROC-AUC 0.961
Serving	FastAPI in Docker, with Docker Compose and Kubernetes (manifest + Helm)
Tooling	scikit-learn, MLflow, pytest, flake8, GitHub Actions, Prometheus, Grafana
Repository	**<add GitHub URL>**
Author	Keerthana S
Date	19 October 2026

Contents

1. Problem, dataset and solution overview
2. Setup and installation
3. Data acquisition and exploratory data analysis
4. Feature engineering and model development
5. Experiment tracking
6. Model packaging and reproducibility
7. CI/CD pipeline and automated testing
8. Containerization and API
9. Production deployment (Kubernetes)
10. Monitoring and logging
11. Architecture, deliverables and repository

1. Problem, dataset and solution overview

Problem. Predict whether a patient has heart disease from routine clinical measurements, and serve that prediction through a monitored REST API that is ready for the cloud.

Dataset. UCI Machine Learning Repository, Heart Disease, file `processed.cleveland.data`. It has 303 rows, 13 input features and the label `num` (0 = no disease, 1–4 = increasing severity). Following the usual convention, `num > 0` is mapped to the binary target `1`.

Feature	Description	Type
age	Age in years	numeric
sex	1 = male, 0 = female	binary
cp	Chest pain type (1 typical angina, 2 atypical, 3 non-anginal, 4 asymptomatic)	nominal
trestbps	Resting blood pressure (mmHg)	numeric
chol	Serum cholesterol (mg/dl)	numeric
fbs	Fasting blood sugar > 120 mg/dl	binary
restecg	Resting ECG result (0–2)	nominal
thalach	Maximum heart rate achieved	numeric
exang	Exercise-induced angina	binary
oldpeak	ST depression induced by exercise	numeric
slope	Slope of the peak exercise ST segment (1–3)	nominal
ca	Number of major vessels coloured by fluoroscopy (0–3)	numeric (count)
thal	3 normal, 6 fixed defect, 7 reversible defect	nominal

Solution at a glance

- **Data:** `src/data.py` downloads, de-duplicates and imputes the data, then writes `data/processed/heart_clean.csv`.
- **Modelling:** a single scikit-learn `Pipeline` (preprocessing + classifier). Logistic Regression and Random Forest are tuned with GridSearchCV and 5-fold stratified cross-validation.
- **Tracking:** MLflow records the parameters, CV and test metrics, ROC and confusion-matrix plots, and the model for every run.
- **Quality gates:** flake8 and 14 pytest unit tests run in GitHub Actions (mirrored locally by `scripts/ci_local.sh`). Any failure stops the pipeline.
- **Serving:** FastAPI `/predict` in a slim Python 3.11 image, run with Docker Compose and on Kubernetes (raw manifest and Helm chart).
- **Observability:** JSON request logs, Prometheus metrics, and a pre-provisioned Grafana dashboard.

Headline results (held-out test set, n = 61)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (selected)	0.885	0.839	0.929	0.881	0.961
Random Forest	0.869	0.813	0.929	0.867	0.945

2. Setup and installation

Prerequisites: Python 3.11, Docker Desktop (with Kubernetes enabled), kubectl, Helm 3, Git.

Repository layout

```
mlops/
├── data/              raw/ (downloaded) · processed/heart_clean.csv
├── notebooks/         01_eda.ipynb · 02_training.ipynb · 03_inference.ipynb
├── src/               config · data · eda · features · train · predict
├── api/main.py        FastAPI service (/predict, /health, /metrics)
├── tests/             pytest unit tests (data, model, API)
├── models/            model.joblib · metrics.json
├── k8s/               deployment.yaml (Namespace, Deployment, LoadBalancer Service)
├── helm/heart-api/    Helm chart (Deployment, Service, optional Ingress)
├── monitoring/        prometheus.yml · Grafana datasource + dashboard
├── .github/workflows/ci.yml
├── scripts/           ci_local.sh · capture_screenshots.py · record_demo.py
├── screenshots/ · video/ · report/
├── Dockerfile · docker-compose.yml · requirements.txt · requirements-api.txt
```

1. Environment and training (clean setup)

```
python -m venv .venv
.venv\Scripts\activate            # Linux/macOS: source .venv/bin/activate
pip install -r requirements.txt

python -m src.data                # download + clean
python -m src.eda                 # EDA figures -> report/figures
python -m src.train               # train + MLflow tracking -> models/model.joblib
pytest && flake8 src api tests     # tests + lint
mlflow ui --backend-store-uri ./mlruns --port 5005
```

2. Run the stack in Docker

```
docker compose up -d --build      # API :8010, Prometheus :9095, Grafana :3005
curl -X POST localhost:8010/predict -H "Content-Type: application/json" -d @sample_request.json
```

3. Deploy to local Kubernetes

```
kubectl --context docker-desktop apply -f k8s/deployment.yaml                  # :8020
helm --kube-context docker-desktop upgrade --install heart-api helm/heart-api \
     -n heart-disease-helm --create-namespace                                   # :8030
```

Access instructions (local)

Endpoint	URL
API, Swagger UI	http://localhost:8010/docs
API on Kubernetes (manifest / Helm)	http://localhost:8020/predict · http://localhost:8030/predict
Prometheus	http://localhost:9095
Grafana (admin / admin)	http://localhost:3005/d/heart-api
MLflow UI	http://localhost:5005

3. Data acquisition and exploratory data analysis

Acquisition and cleaning (`src/data.py`)

- The file is downloaded from the UCI archive with `requests` and cached in `data/raw/`. The step is idempotent.
- `?` is parsed as missing. Only **ca (4 rows)** and **thal (2 rows)** have gaps. Both are discrete codes, so they are filled with the mode.
- Duplicates are dropped, types are coerced to integers, and `num` becomes the binary `target`.
- Result: 303 rows with no missing values. **164 no disease / 139 disease** (46% positive).

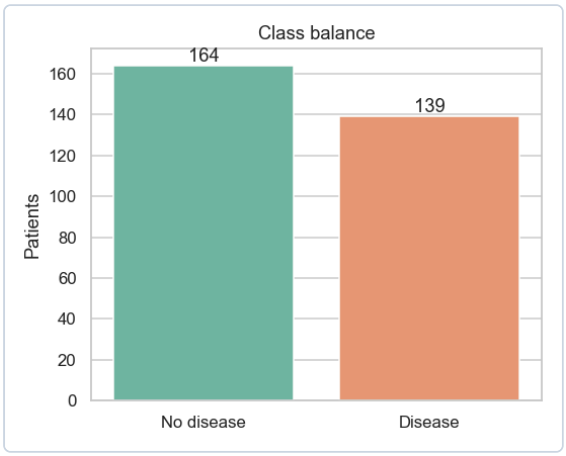


Fig 1. Class balance: close to even, so no resampling is needed

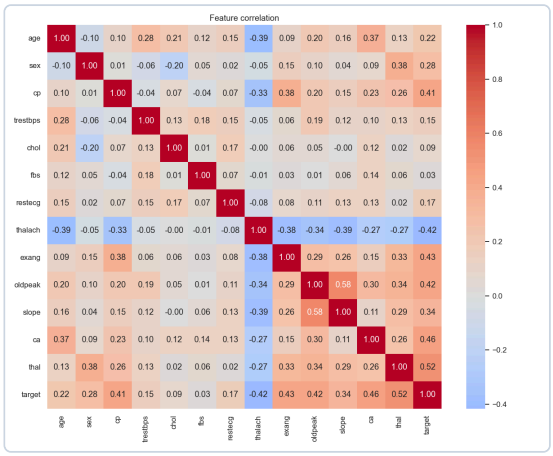


Fig 2. Correlation heatmap

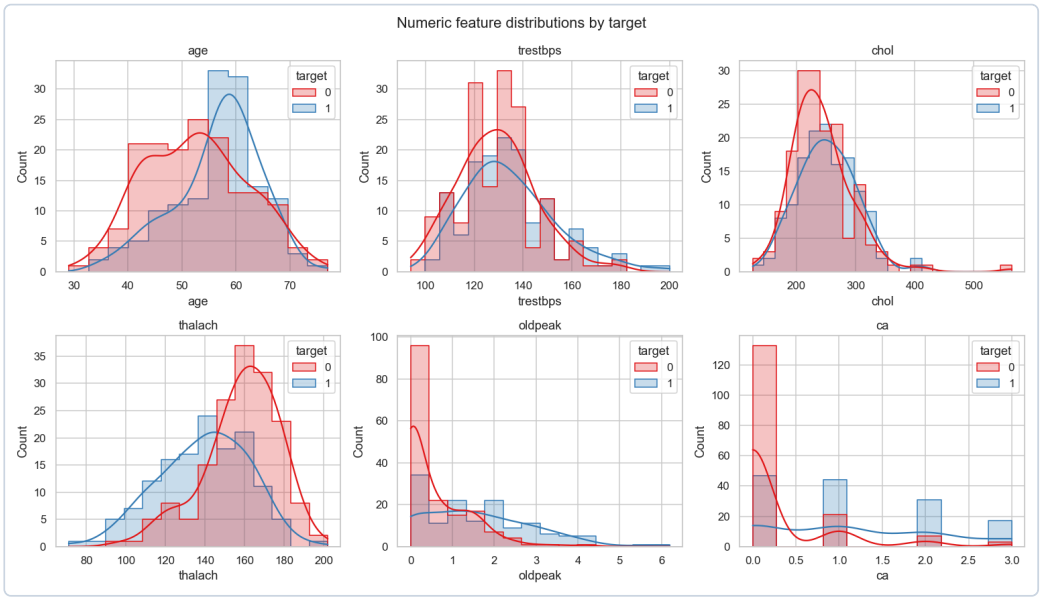


Fig 3. Numeric feature distributions by target (0 = no disease, 1 = disease)

Key findings

- Strongest correlations with the target: **thal 0.52, ca 0.46, exang 0.43, oldpeak 0.42, thalach −0.42, cp 0.41**.
- Patients with disease reach a lower maximum heart rate (mean 139 vs 158 bpm), show more ST depression (1.6 vs 0.6) and are slightly older (56.6 vs 52.6 years).
- Cholesterol (0.09) and fasting blood sugar (0.03) carry little signal on their own.
- `cp`, `thal`, `restecg` and `slope` are nominal codes, so they are one-hot encoded rather than treated as ordered numbers.

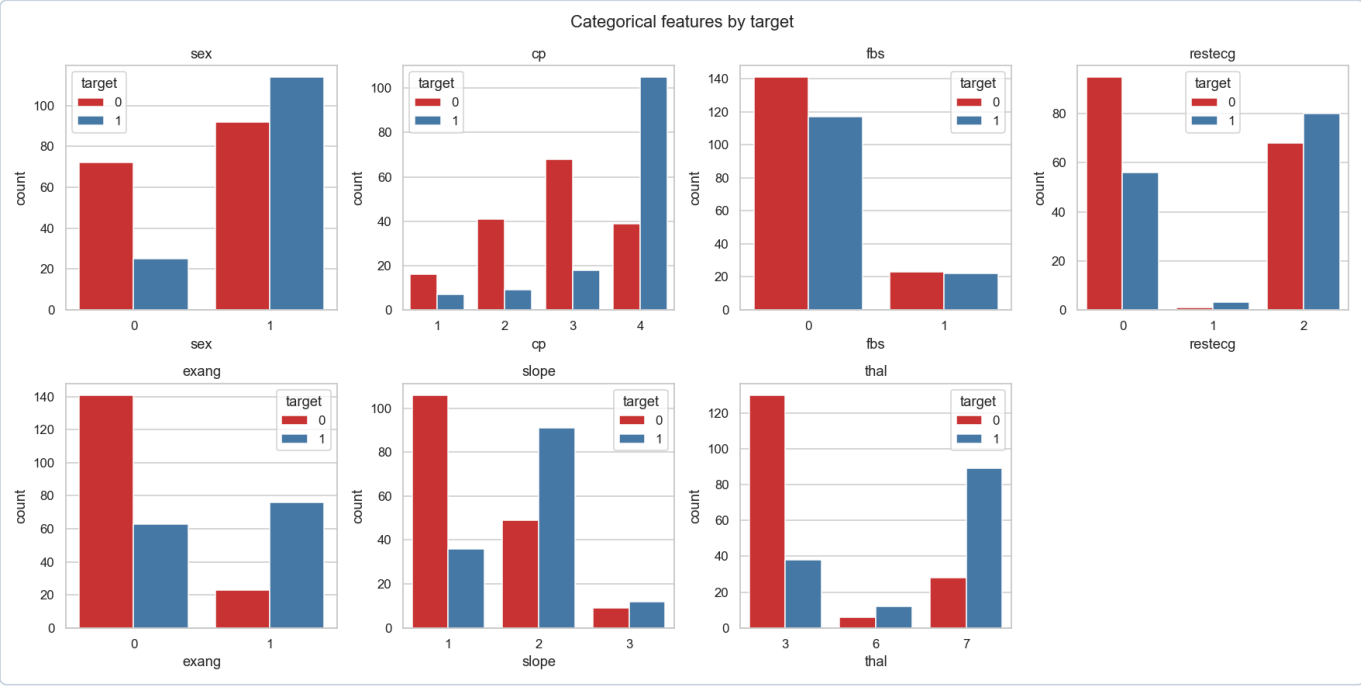


Fig 4. Categorical features by target. Asymptomatic chest pain (cp = 4), reversible defect (thal = 7) and exercise angina stand out.

4. Feature engineering and model development

Preprocessing (`src/features.py`)

Group	Columns	Steps
Numeric	age, trestbps, chol, thalach, oldpeak, ca	SimpleImputer(median) → StandardScaler
Categorical	sex, cp, fbs, restecg, exang, slope, thal	SimpleImputer(most_frequent) → OneHotEncoder(handle_unknown="ignore")

This gives 25 engineered features. The `ColumnTransformer` sits inside the same `Pipeline` as the classifier, so it is fitted only on training folds (no leakage) and the identical transform runs at serving time.

Model selection and tuning (`src/train.py`)

- Stratified 80/20 train/test split (seed 42). Tuning uses only the training set.
- GridSearchCV with 5-fold StratifiedKFold, optimising ROC-AUC.
- **Logistic Regression:** C ∈ {0.01, 0.1, 1, 10}, L2 penalty. Best C = 0.1 (strong regularisation suits n = 242).
- **Random Forest:** n_estimators ∈ {100, 300}, max_depth ∈ {None, 4, 8}, min_samples_leaf ∈ {1, 3}. Best: 100 trees, unlimited depth, min_samples_leaf = 3.
- The best model is chosen by **cross-validated ROC-AUC**, not by the small 61-row test set.

Evaluation

Model	CV Acc	CV Prec	CV Rec	CV F1	CV ROC-AUC	Test Acc	Test ROC-AUC
Logistic Regression	0.851	0.878	0.792	0.830	**0.911**	0.885	0.961
Random Forest	0.818	0.813	0.792	0.798	0.900	0.869	0.945

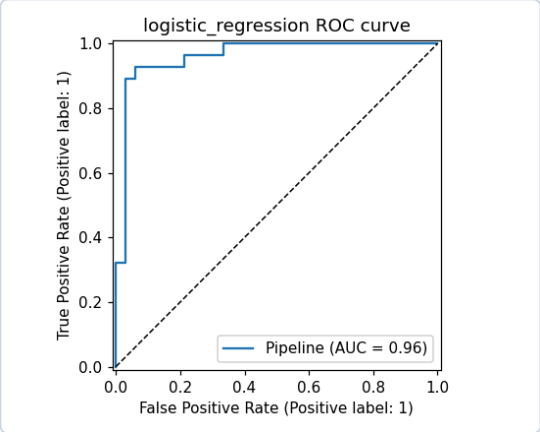


Fig 5. Logistic Regression ROC curve (test)

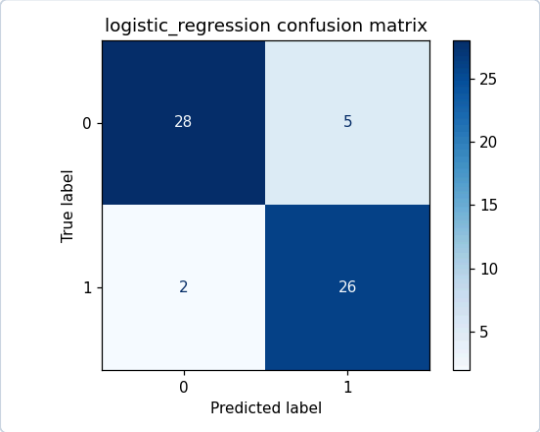


Fig 6. Logistic Regression confusion matrix (test)

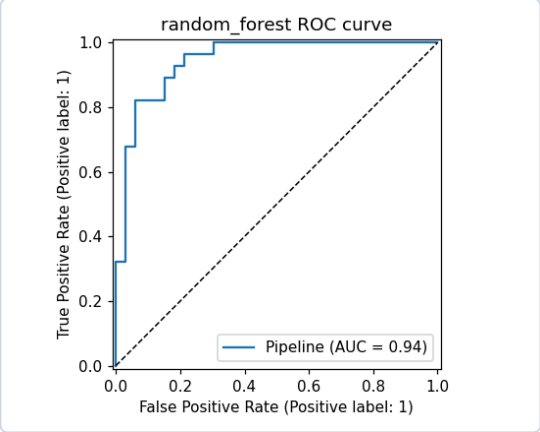


Fig 7. Random Forest ROC curve (test)

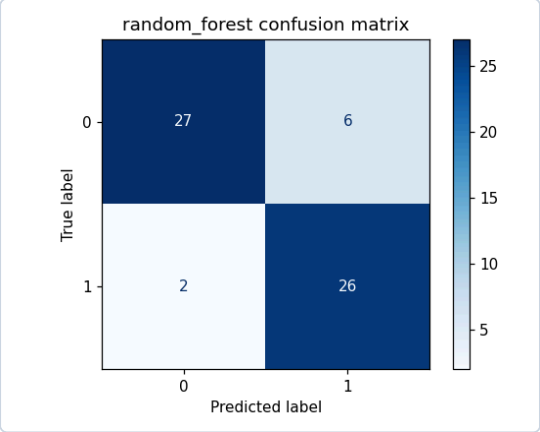


Fig 8. Random Forest confusion matrix (test)

Why Logistic Regression. It scores higher on every cross-validated metric. Its coefficients are directly interpretable, which matters in a clinical setting. It is also tiny and fast to serve. On only 303 rows, the Random Forest's extra capacity adds variance without improving accuracy. Test recall of 0.93 means 26 of 28 disease cases are caught, which is the priority for a screening tool.

5. Experiment tracking

MLflow (file store `./mlruns`, experiment `heart-disease`) logs one run per candidate model for each training execution:

- **Parameters:** model type and the best hyperparameters from the grid search.
- **Metrics:** 5 CV metrics (accuracy, precision, recall, F1, ROC-AUC) and the same 5 on the test set.
- **Artifacts:** ROC curve and confusion matrix PNGs, plus the full sklearn pipeline as an MLflow model with an input example.

Training ran three times (CLI, notebook, local CI) with identical results, which confirms reproducibility with fixed seeds.

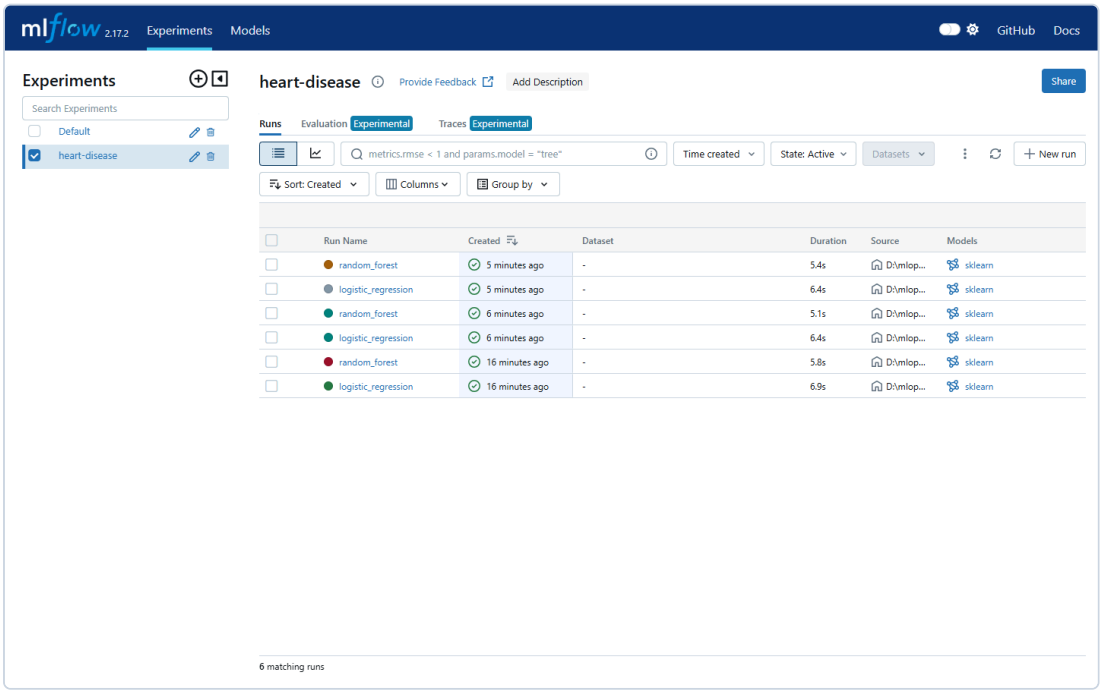


Fig 9. MLflow experiment with all tracked runs

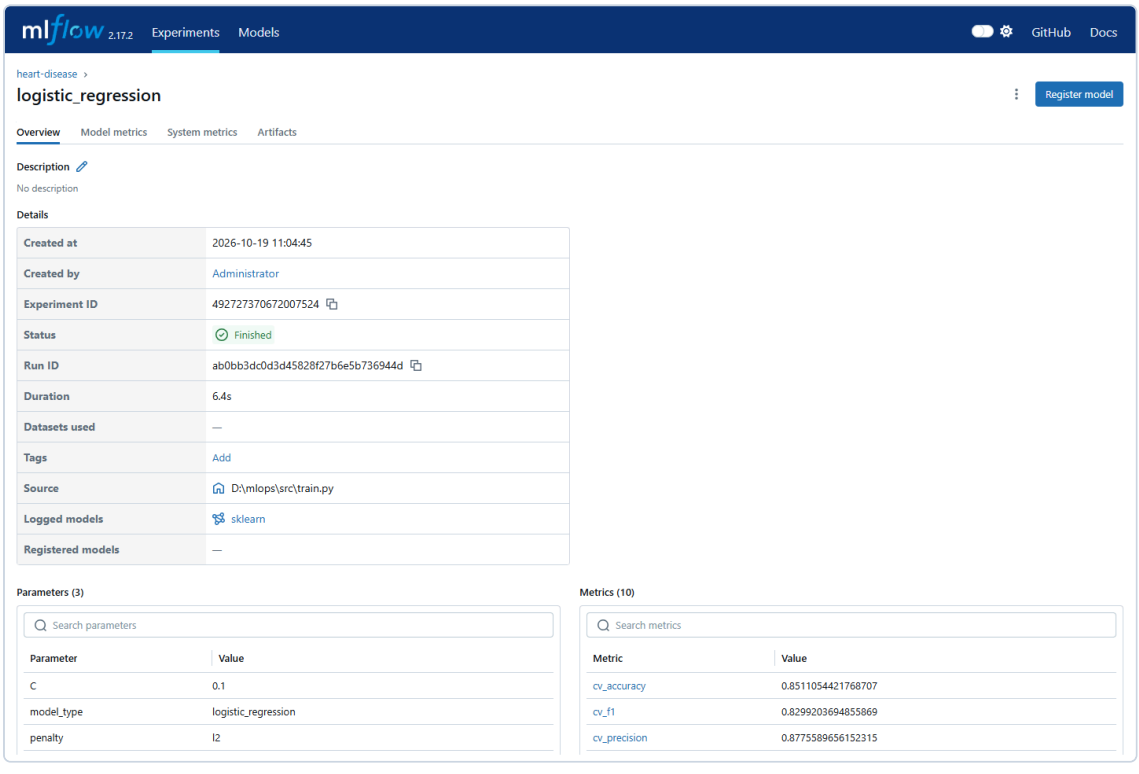


Fig 10. Run detail: parameters and metrics for the selected Logistic Regression

6. Model packaging and reproducibility

- **Format:** the fitted `Pipeline` (preprocessing + model) is saved to `models/model.joblib`. The same pipeline is also logged as an MLflow model, with `MLmodel`, `conda.yaml` and `requirements.txt` generated by MLflow.
- **Metadata:** `models/metrics.json` stores the selected model, the best parameters, the run IDs and all metrics.
- **Environment:** every dependency is pinned in `requirements.txt` (training) and `requirements-api.txt` (serving).
- **Determinism:** one `RANDOM_STATE = 42` covers the split, the CV folds and the models. The column lists and paths are defined once in `src/config.py`.
- **Same code path:** the API calls `src.predict.predict()`, the same function the notebooks and CLI use, so training/serving skew is impossible by construction.

7. CI/CD pipeline and automated testing

Unit tests (`tests/`, 14 tests)

File	What is verified
test_data.py	no missing values after cleaning, target binarisation, all features kept, duplicate removal, input not mutated
test_model.py	preprocessor output shape and no NaNs, unseen category handled, predict output contract, end-to-end train() with MLflow into a temp dir
test_api.py	/health, /predict returns a prediction and confidence plus a request id, 422 on a missing field, 422 on an out-of-range value, /metrics exposed

GitHub Actions (`.github/workflows/ci.yml`)

Triggered on push to `main`, on pull requests, and manually. Job **lint-test-train**: checkout → Python 3.11 with pip cache → install → **flake8** → **pytest** (JUnit XML) → download data → EDA → train. It uploads `test-results.xml`, `train.log`, `models/`, the figures and `mlruns/` as workflow artifacts, even on failure. Job **docker** (needs the first job): builds the image from the trained model, runs it, and smoke-tests `/health` and `/predict`, failing if the response lacks `confidence`. Container logs are always printed.

`scripts/ci_local.sh` runs the same steps locally with `set -euo pipefail`.

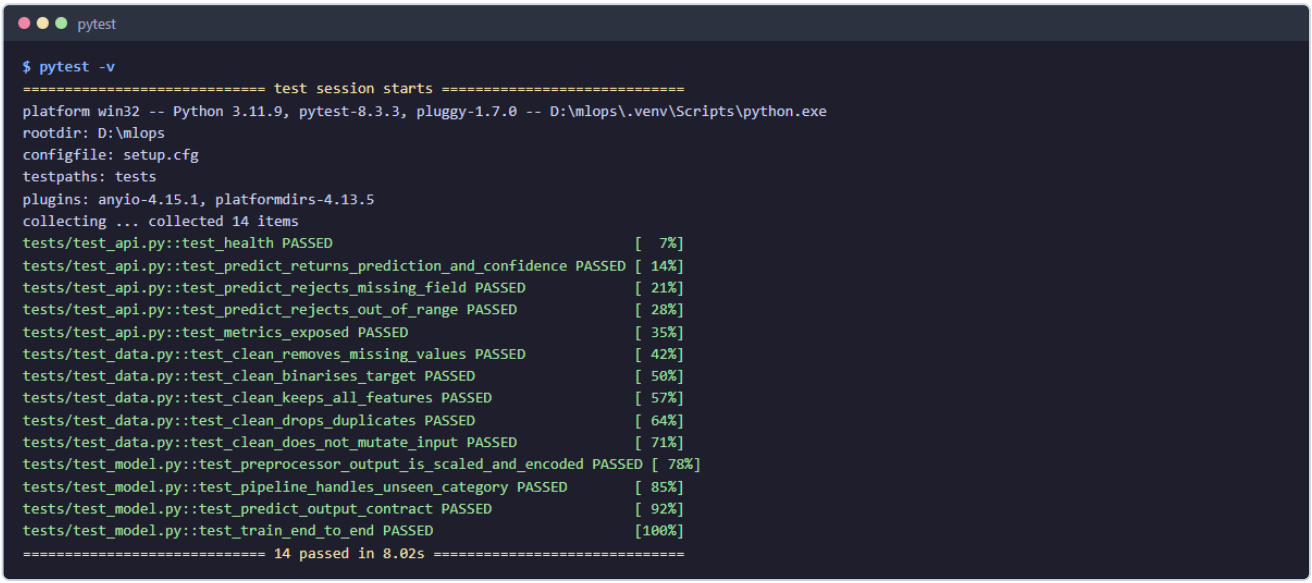


Fig 11. pytest: 14 passed

ci pass

```
$ PY=.venv/Scripts/python bash scripts/ci_local.sh

===== [11:04:28] Lint (flake8) =====
flake8: no issues

===== [11:04:29] Unit tests (pytest) =====
============================= test session starts =============================
platform win32 -- Python 3.11.9, pytest-8.3.3, pluggy-1.7.0
rootdir: D:\mlops
configfile: setup.cfg
testpaths: tests
plugins: anyio-4.15.1, platformdirs-4.13.5
collected 14 items

tests\test_api.py .....                                                  [ 35%]
tests\test_data.py .....                                                 [ 71%]
tests\test_model.py ....                                                 [100%]

---------------- generated xml file: D:\mlops\test-results.xml ----------------
============================== 14 passed in 8.08s ==============================

===== [11:04:38] Download & clean data =====
INFO Raw data already present at D:\mlops\data\raw\processed.cleveland.data
INFO Saved 303 rows to D:\mlops\data\processed\heart_clean.csv (positive rate 0.46)

===== [11:04:39] EDA figures =====
wrote D:\mlops\report\figures\class_balance.png
wrote D:\mlops\report\figures\numeric_histograms.png
wrote D:\mlops\report\figures\categorical_counts.png
wrote D:\mlops\report\figures\correlation_heatmap.png

===== [11:04:42] Train & track (MLflow) =====
INFO logistic_regression best={'model__C': 0.1, 'model__penalty': 'l2'} cv_roc_auc=0.911 test_roc_auc=0.961
INFO random_forest best={'model__max_depth': None, 'model__min_samples_leaf': 3, 'model__n_estimators': 100} cv_roc_auc=0.900 test_roc_auc=0.945
INFO Selected logistic_regression -> D:\mlops\models\model.joblib

===== [11:04:58] Docker build =====
sha256:7238dd5546d4f826ea801672e88a235c4fe9917ab235be8abdb0c363623c8bc3

===== [11:04:58] Container smoke test =====
{"status":"ok","model_version":"1.0.0"}
{"prediction":1,"label":"disease","probability_disease":0.9478,"confidence":0.9478,"model_version":"1.0.0"}

===== [11:05:03] PIPELINE PASSED =====
```

Fig 12. Full pipeline run: every stage passes

ci fail

```
$ bash scripts/ci_local.sh   # with a lint error injected

===== [11:05:25] Lint (flake8) =====
tests\test_zz_failure_demo.py:1:1: F401 'os' imported but unused
exit=1

$ bash scripts/ci_local.sh   # with a failing unit test injected
tests\test_data.py .....                                                 [ 66%]
tests\test_model.py ....                                                 [ 93%]
tests\test_zz_failure_demo.py F                                          [100%]
================================== FAILURES ===================================
_____________________________ test_broken _____________________________
    def test_broken():
>       assert 1 == 2
E       assert 1 == 2
tests\test_zz_failure_demo.py:2: AssertionError
---------------- generated xml file: D:\mlops\test-results.xml ----------------
=========================== short test summary info ===========================
FAILED tests/test_zz_failure_demo.py::test_broken - assert 1 == 2
======================== 1 failed, 14 passed in 8.70s =========================
exit=1
```

Fig 13. Fail-fast: an injected lint error, and separately a failing test, each stop the pipeline with exit code 1 and clear logs

> GitHub Actions run screenshots are added after the repository is pushed and the first workflow run finishes.

8. Containerization and API

- **Image:** `python:3.11-slim`, runtime dependencies only, non-root `appuser`, Docker `HEALTHCHECK` on `/health`, model baked in at `/app/models/model.joblib`.
- **Endpoints:** `POST /predict` (JSON in; prediction, label, probability and confidence out), `GET /health`, `GET /metrics` (Prometheus), `/docs` (Swagger).
- **Validation:** a Pydantic schema with range checks on every field. Bad input gets `422` with a clear message.
- Each response carries an `x-request-id` header for tracing.

```
POST /predict  {"age":63,"sex":1,"cp":4,"trestbps":145,"chol":233,"fbs":1,"restecg":2,
                "thalach":150,"exang":1,"oldpeak":2.3,"slope":3,"ca":2,"thal":7}
→ 200
{"prediction":1,"label":"disease","probability_disease":0.9478,"confidence":0.9478,"model_version":"1.0.0"}
```

docker compose

```
$ docker compose ps
NAME                      IMAGE                      STATUS                   PORTS
heart-mlops-api-1         heart-disease-api:latest   Up 8 minutes (healthy)   0.0.0.0:8010->8000/tcp, [::]:8010->8000/tcp
heart-mlops-grafana-1     grafana/grafana:11.3.0     Up 8 minutes             0.0.0.0:3005->3000/tcp, [::]:3005->3000/tcp
heart-mlops-prometheus-1  prom/prometheus:v2.55.1    Up 8 minutes             0.0.0.0:9095->9090/tcp, [::]:9095->9090/tcp

$ docker images heart-disease-api
IMAGE                      ID            DISK USAGE   CONTENT SIZE   EXTRA
heart-disease-api:ci       7238dd5546d4       715MB          164MB
heart-disease-api:latest   1b3256e8539f       715MB          164MB   U
```

Fig 14. Docker Compose stack: API (healthy), Prometheus, Grafana

curl predict

```
$ curl localhost:8010/health
{"status":"ok","model_version":"1.0.0"}

$ curl -X POST localhost:8010/predict -H "Content-Type: application/json" -d @sample_request.json
{"prediction":1,"label":"disease","probability_disease":0.9478,"confidence":0.9478,"model_version":"1.0.0"}

$ curl -X POST localhost:8010/predict -d {"age": 63}     # invalid input
{"detail":[{"type":"missing","loc":["body","sex"],"msg":"Field required","input":{"age":63}},{"type":"missing","loc":["body","cp"],"msg":"Field require
d","input":{"age":63}},{"type":"missing","loc":["body","trestbps"],"msg":"Field required","input":{"age":63}},{"type":"missing","loc":["body","cho
l"],"msg":"Field required","input":{"age":63}},{"type":"missing","loc":["body","fbs"],"msg":"Field requ ...
```

Fig 15. Container proof: health check, a valid prediction, and a rejected invalid request

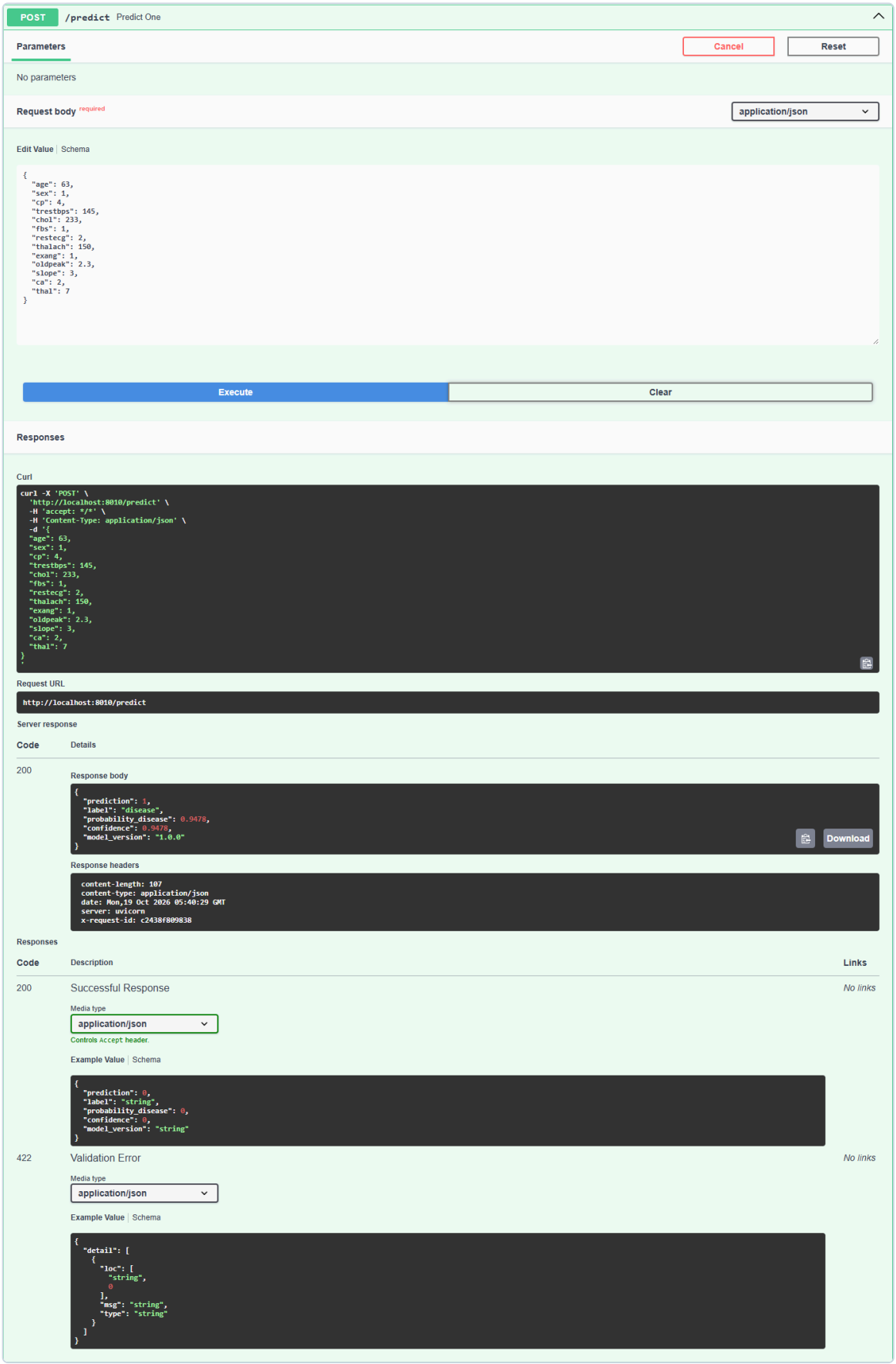


Fig 16. Swagger UI: /predict executed against the container

9. Production deployment (Kubernetes)

The target is the local Kubernetes cluster that ships with Docker Desktop (context `docker-desktop`). The Docker image is used directly with `imagePullPolicy: IfNotPresent`. Two equivalent deployment methods are provided and both are running:

	Raw manifest	Helm chart
Source	`k8s/deployment.yaml`	`helm/heart-api` (values.yaml)
Namespace	heart-disease	heart-disease-helm
Replicas	2	2 (configurable)
Exposure	LoadBalancer → localhost:8020	LoadBalancer → localhost:8030 (Ingress optional)

Both use readiness and liveness probes on `/health`, CPU and memory requests and limits, and `prometheus.io/scrape` annotations. The Helm chart passes `helm lint`. Moving to a cloud cluster (GKE/EKS/AKS) only means pushing the image to a registry and changing `image.repository`.

k8s manifest

```
$ kubectl --context docker-desktop -n heart-disease get deploy,pods,svc
NAME                         READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/heart-api    2/2     2            2           8m8s

NAME                             READY   STATUS    RESTARTS   AGE
pod/heart-api-594455469d-cgjjb   1/1     Running   0          8m8s
pod/heart-api-594455469d-rmkmg   1/1     Running   0          8m8s

NAME                 TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
service/heart-api    LoadBalancer   10.96.233.154   172.18.0.5    8020:30537/TCP   8m8s

$ curl -X POST localhost:8020/predict -d @sample_request.json
{"prediction":1,"label":"disease","probability_disease":0.9478,"confidence":0.9478,"model_version":"1.0.0"}
```

Fig 17. Manifest deployment: 2/2 pods running, LoadBalancer, endpoint verified

k8s helm

```
$ helm --kube-context docker-desktop list -A
NAME            NAMESPACE               REVISION        UPDATED                                 STATUS          CHART           APP VERSION
heart-api       heart-disease-helm      1               2026-10-19 11:02:56.1944004 +0530 IST   deployed        heart-api-0.1.0 1.0.0

$ kubectl --context docker-desktop -n heart-disease-helm get pods,svc
NAME                             READY   STATUS    RESTARTS   AGE
pod/heart-api-594455469d-b4pfv   1/1     Running   0          3m33s
pod/heart-api-594455469d-bzzxf   1/1     Running   0          3m33s

NAME                 TYPE           CLUSTER-IP     EXTERNAL-IP   PORT(S)          AGE
service/heart-api    LoadBalancer   10.96.223.67   172.18.0.6    8030:31289/TCP   3m33s

$ curl -X POST localhost:8030/predict -d @sample_request.json
{"prediction":1,"label":"disease","probability_disease":0.9478,"confidence":0.9478,"model_version":"1.0.0"}
```

Fig 18. Helm release deployed and verified

10. Monitoring and logging

- **Logging:** an HTTP middleware writes one JSON line per request (request_id, method, path, status, latency_ms). Each prediction also logs its input and output. The logs go to stdout, so `docker logs` or `kubectl logs` collect them.
- **Metrics:** `api_requests_total{method,endpoint,status}`, the histogram `api_request_latency_seconds{endpoint}`, and `model_predictions_total{label}` (watches for drift in the class mix).
- **Prometheus** scrapes `api:8000/metrics` every 5 s. **Grafana** is provisioned automatically with the data source and the "Heart Disease API" dashboard.



Fig 19. Structured JSON request and prediction logs

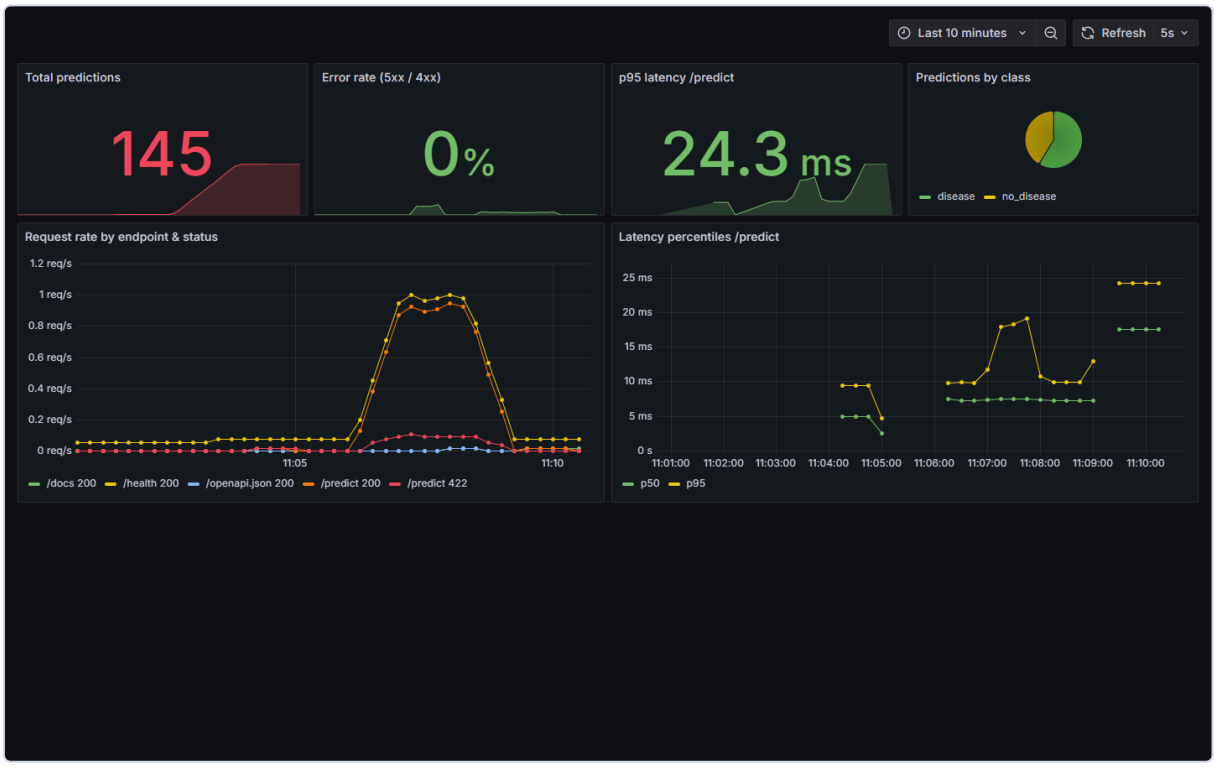


Fig 20. Grafana: total predictions, error rate, p95 latency, class mix, request rate and latency percentiles

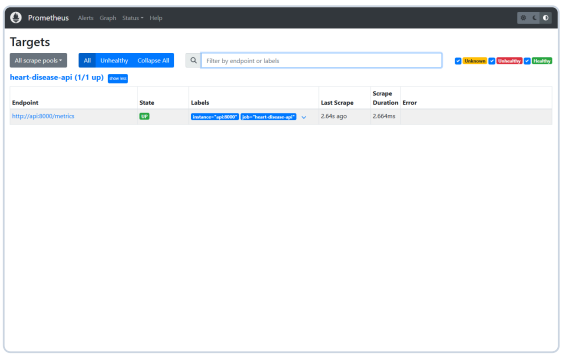


Fig 21. Prometheus target healthy

11. Architecture, deliverables and repository

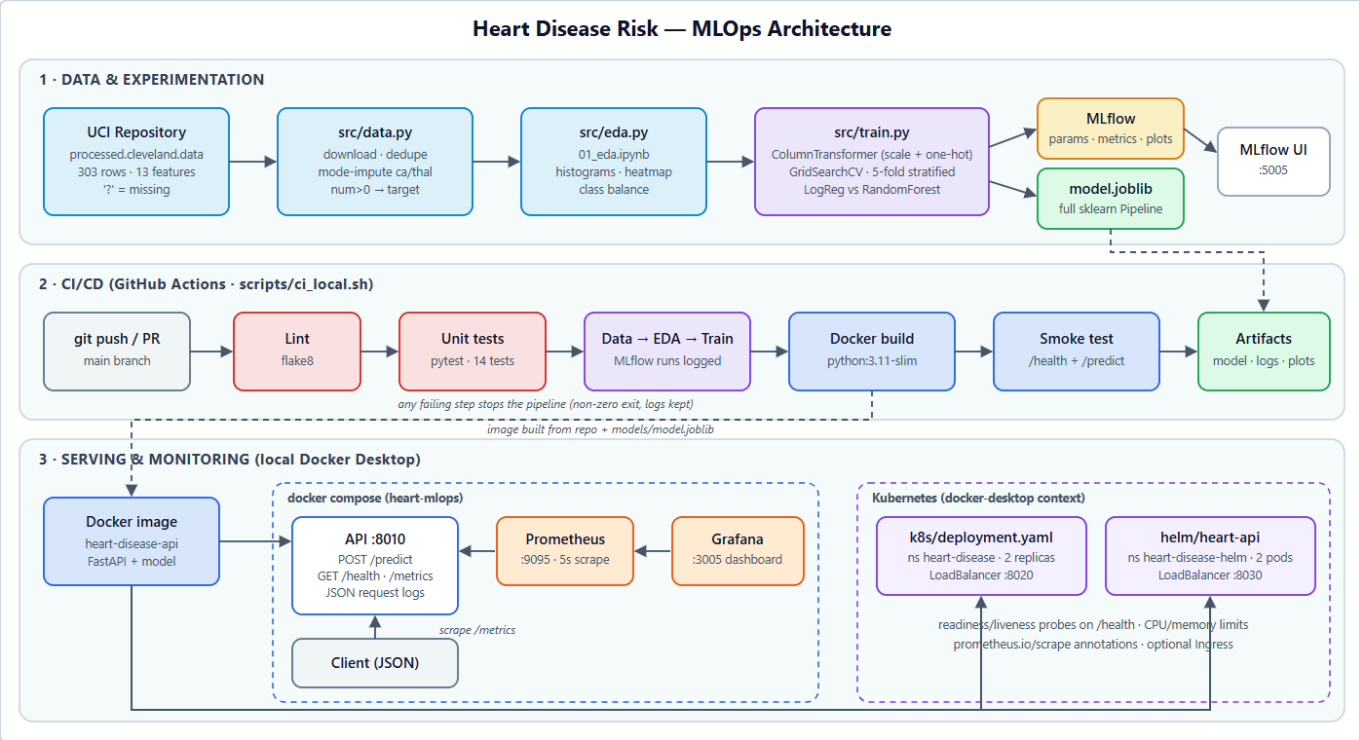


Fig 22. End-to-end architecture

Deliverables checklist

Deliverable	Location
Code, Dockerfile, requirements	`src/`, `api/`, `Dockerfile`, `docker-compose.yml`, `requirements*.txt`
Cleaned dataset + download script	`data/processed/heart_clean.csv`, `src/data.py`
Notebooks (EDA, training, inference)	`notebooks/01_eda.ipynb`, `02_training.ipynb`, `03_inference.ipynb`
Unit tests	`tests/` (14 tests)
CI workflow	`.github/workflows/ci.yml`, `scripts/ci_local.sh`
Deployment manifests / Helm	`k8s/deployment.yaml`, `helm/heart-api/`
Screenshots	`screenshots/`
Report	`report/report.pdf`
Access instructions	Section 2 and `README.md`
Pipeline video	`video/pipeline_demo.webm`

Repository

<add GitHub URL>

Limitations and next steps

- The dataset is small (303 rows) and comes from one site. Validating on the Hungarian, Swiss and VA subsets would test how well the model generalises.
- Possible next steps: an MLflow Model Registry with staged promotion, pushing images to a container registry from CI, data-drift detection on the logged inputs, and HPA autoscaling.